Java SE 17 Programmer II（1Z0-826-JPN）　試験内容 2 関数型インタフェースとラムダ式　#05

ラムダ式と関数型インタフェース

関数型インタフェース・ラムダ式の書き方・メソッド参照・java.util.function

各ページの下に、根拠にした資料（Oracle の試験内容チェックリスト、Java 言語仕様 JLS、Java SE 17 API の java.util.function、The Java Tutorials の Lambda Expressions・Method References）を書いた。コードの結果は JDK 17（Eclipse Temurin 17.0.21）で実行して確かめた

全体像 「抽象メソッド 1 つ」に処理を渡す

関数型インタフェース
抽象メソッドがちょうど 1 つ

↑ ターゲット型として代入

ラムダ式
s -> s.length()

メソッド参照
String::length

ラムダ式もメソッド参照も、それだけでは型が決まらない。代入先・引数・キャストの型（ターゲット型）で決まる。

java.util.function の 4 つの基本

型	メソッド	形
Supplier<T>	get()	() → T
Consumer<T>	accept(T)	T → void
Predicate<T>	test(T)	T → boolean
Function<T,R>	apply(T)	T → R

メソッド名の取り違え（Supplier の apply など）は cannot find symbol。

出典：JLS 9.8・15.13・15.27／API java.util.function／試験内容チェックリスト 2

第1章

関数型インタフェース

抽象メソッドの数え方。

抽象メソッドの数え方

```
@FunctionalInterface
interface Calc {
  int apply(int a, int b);          // 1 つ目（これだけ）
  default int twice(int x) { return apply(x, 2); }
  static Calc plus() { return (a, b) -> a + b; }
  boolean equals(Object o);         // 数えない
  String toString();                // 数えない
}
Calc c = (a, b) -> a * b;
c.apply(3, 4)  // 12
c.twice(5)     // 10
```

中身	数えるか
ふつうの抽象メソッド	○
default・static・private	×
Object の public と同じ	×
親 2 つから同じ go(String)	1 つと数える

`interface D extends Runnable { void stop(); }` は run と stop で 2 つ → @FunctionalInterface でエラー。

@FunctionalInterface は付けなくても使える。付けると、条件違反をコンパイラが見つける。

出典：JLS 9.8・9.6.4.9、API java.lang.FunctionalInterface／JDK 17 でのコンパイルと実行の結果

ターゲット型がないとエラー

```
Object o = () -> System.out.println("x");
// × Object is not a functional interface
var r = () -> System.out.println("x");
// × cannot infer type for local variable r
//   (lambda expression needs an explicit target-type)
Object o = (Runnable) () -> System.out.println("x");
// ○ キャストで型を与える
Runnable r = () -> { return 1; };
// × bad return type ... unexpected return value
```

オーバーロードの選び分け

```
static void run(Runnable r)       { print("R"); }
static void run(Supplier<String> s) { print("S"); }
run(() -> "x");                   // S
run(() -> System.out.print(""));  // R
```

> 値を返すラムダ式は void 型（Runnable）に合わない。void のメソッド呼び出しは Supplier に合わない。

出典：JLS 15.27.3 Type of a Lambda Expression・14.4.1・15.12.2.1／JDK 17 でのコンパイルと実行の結果

第2章

ラムダ式の書き方

引数・本体・スコープ。

引数と本体の書き方

書き方	可否
x -> x + 1	○
(x, y) -> x + y	○
(Integer x, Integer y) -> x + y	○
(var x, var y) -> x + y	○ Java 11〜
(Integer x, y) -> x + y	× 混在
Integer x -> x + 1	× ( ) が必要
(int x) -> x + 1 ※Function<Integer,Integer>	× 型が違う

本体

```
() -> "hi"                 // ○ 式
() -> { return "hi"; }     // ○ ブロック＋return
() -> { "hi"; }            // × not a statement
() -> return "hi";         // × { } が必要
```

混在のエラー文：cannot mix implicitly-typed and explicitly-typed parameters。

int と書くと incompatible parameter types in lambda expression。

出典：JLS 15.27.1 Lambda Parameters・15.27.2 Lambda Body／JDK 17 でのコンパイル結果

ラムダ式のスコープ

```
String s = "outer";
Function<String,String> f = s -> s.toUpperCase();
// × variable s is already defined in method main(String[])

int x = 5;
Runnable r = () -> { int x = 10; };
// × variable x is already defined
```

> ラムダ式の本体は新しいスコープを作らない。外側と同じ名前は使えない。

	ラムダ式	無名クラス
外側と同じ名前の変数	×	○（覆い隠す）
this	外側のインスタンス	無名クラス自身
外側のローカル変数	effectively final のみ（#04）	

ラムダ式を返すラムダ式

```
Function<Integer, Function<Integer,Integer>> adder
    = a -> b -> a + b;
adder.apply(3).apply(4)   // 7
```

出典：JLS 15.27.2・6.4 Shadowing and Obscuring・15.8.3／JDK 17 でのコンパイルと実行の結果

第3章

メソッド参照

4 つの種類と、ターゲット型による選ばれ方。

メソッド参照の 4 種類

種類	例	同じ意味のラムダ式	引数の使われ方
static メソッド	Integer::parseInt	s -> Integer.parseInt(s)	全部がメソッドの引数
束縛された（特定のオブジェクト）	prefix::concat	s -> prefix.concat(s)	全部がメソッドの引数
束縛されていない（任意のオブジェクト）	String::startsWith	(a, b) -> a.startsWith(b)	1 つ目が呼ぶ相手、残りが引数
コンストラクタ	ArrayList::new／int[]::new	() -> new ArrayList<>()／n -> new int[n]	全部がコンストラクタの引数

```
Comparator<String> c = String::compareTo;
c.compare("b", "a")          // 1 ("b".compareTo("a"))
Supplier<Integer> s = String::length;
// × invalid method reference
//   unexpected instance method length() found in unbound lookup
String w = "hey";
Supplier<Integer> t = w::length;   // ○ 3
```

コンストラクタ参照の get・apply は、呼ぶたびに新しいオブジェクトを作る（l1 == l2 は false）。

レコードの正規コンストラクタも `Sq::new` で参照できる。

出典：JLS 15.13・15.13.1・15.13.3／The Java Tutorials「Method References」／JDK 17 でのコンパイルと実行の結果

同じ参照でも、ターゲット型で中身が変わる

```
Consumer<String> c = System.out::println;
Runnable r          = System.out::println;
c.accept("hi");   // println(String) → hi
r.run();          // println() → 空の行
```

```
BiFunction<Integer,Integer,Integer> f = Math::max;
BinaryOperator<Integer> g = Integer::sum;
ToIntBiFunction<Integer,Integer> h = Integer::compare;
f.apply(2,9) + g.apply(2,9) + h.applyAsInt(2,9)
// 9 + 11 + (-1) = 19（文字列がないので足し算）
```

関数記述子（引数の型 → 戻り値の型）

↓ これに合うメソッドを探す

String::concat
(String, String) → String ＝ BinaryOperator<String> ○

String::length
String → int ＝ Function / ToIntFunction ○、Supplier ×

> Integer.compare は −1・0・1 を返す（差ではない）。

出典：JLS 15.13.1、API PrintStream.println・Math.max・Integer.sum・Integer.compare／JDK 17 での実行結果

第4章

java.util.function

派生したインタフェースと、合成のメソッド。

派生したインタフェース

型	メソッド	形
UnaryOperator<T>	apply	T → T
BinaryOperator<T>	apply	(T, T) → T
BiFunction<T,U,R>	apply	(T, U) → R
BiConsumer<T,U>	accept	(T, U) → void
BiPredicate<T,U>	test	(T, U) → boolean

Bi の Supplier はない。BinaryOperator には static の minBy・maxBy。

基本データ型用（ボクシングしない）

型	メソッド	形
IntSupplier	getAsInt	() → int
BooleanSupplier	getAsBoolean	() → boolean
IntPredicate	test	int → boolean
IntFunction<R>	apply	int → R
ToIntFunction<T>	applyAsInt	T → int
IntUnaryOperator	applyAsInt	int → int

戻り値が基本データ型になるものは getAs〜・applyAs〜。

出典：API java.util.function（UnaryOperator・BinaryOperator・BiFunction・BiConsumer・BiPredicate・IntSupplier ほか）

合成のメソッド

Function：andThen と compose

```
plus1 = x -> x + 1;  times2 = x -> x * 2;
plus1.andThen(times2).apply(5)  // (5+1)*2 = 12
plus1.compose(times2).apply(5)  // 5*2+1  = 11
Function.identity().apply(5)    // 5
```

Consumer：andThen（同じ引数で左から順）

```
a.andThen(b).andThen(a).accept("ab");
// a:"AB"  b:2  a:"AB"  → AB2AB
```

Predicate：and・or・negate・not・isEqual

```
p = empty.negate().and(longer.negate())
         .or(s -> s.startsWith("z"));
"" → false  "ab" → true  "abcd" → false  "zzzzz" → true
Predicate.not(String::isBlank).test("  ")  // false
Predicate.isEqual("a").test(null)          // false（NPE なし）
```

> `p.and(null)` は合成した時点で NullPointerException。

出典：API Function.andThen・compose・identity、Consumer.andThen、Predicate.and・or・negate・not・isEqual／JDK 17 での実行結果

コレクションの関数型メソッド

```
List<String> list = new ArrayList<>(List.of("a","bb","ccc"));
list.removeIf(s -> s.length() == 2);   // Predicate
list.replaceAll(String::toUpperCase);  // UnaryOperator
list.forEach(System.out::print);       // Consumer → ACCC
Map<String,Integer> m = new TreeMap<>(Map.of("x",1,"y",2));
m.replaceAll((k, v) -> v * 10);        // BiFunction
m.forEach((k, v) -> print(k + v));     // BiConsumer → x10y20
```

メソッド	引数の型
Collection.removeIf	Predicate
List.replaceAll	UnaryOperator
Iterable.forEach	Consumer
Map.forEach	BiConsumer
Map.replaceAll	BiFunction
Map.computeIfAbsent	Function

これらはストリームを使わずに、コレクションそのものを変える（#06〜で扱うストリームとの違い）。

出典：API Collection.removeIf・List.replaceAll・Iterable.forEach・Map.forEach・Map.replaceAll・Map.computeIfAbsent／JDK 17 での実行結果

本番での問われ方

ねらい	見るところ
関数型インタフェースか	抽象メソッドの数（default・static・Object のメソッドは数えない、継承分を足す）
ラムダ式がコンパイルできるか	型あり・なし・var の混在、( ) と { } と return、外側と同じ名前、effectively final
ターゲット型	Object・var への代入は×、キャストで○、void か値を返すか
メソッド参照	4 種類のどれか、1 つ目の引数が呼ぶ相手になるか、Supplier に unbound は×
メソッド名	get・accept・test・apply、getAsInt・applyAsInt
合成の順	andThen は左から、compose は右から、null は合成時に NPE

出典：試験内容チェックリスト 2／本資料の問題演習（全 30 問、JDK 17 で確認）